

TP - Extraction d'informations avec un petit LLM

Construction d'une mini-application locale et expérimentation du prompting

Objectif : développer une application capable d'extraire des informations structurées à partir d'un texte en utilisant un petit modèle de langage local, puis mesurer l'effet de différentes stratégies de prompting.

1. Contexte du TP

Vous utiliserez obligatoirement un petit LLM d'environ 1 milliard de paramètres, par exemple Llama 3.2 1B. Le modèle devra être exécuté localement via un serveur comme Ollama ou llama.cpp.

L'application recevra un texte en entrée et devra retourner les entités et relations factuelles qui y sont présentes, sous une forme structurée et exploitable par un programme.

Exemple

Texte d'entrée
Alice Martin studied at Stanford University. After graduating, she joined Google in 2018.
Sortie attendue (exemple)
<pre>{ "relations": [{"subject": "Alice Martin", "relation": "educated_at", "object": "Stanford University"}, {"subject": "Alice Martin", "relation": "employed_by", "object": "Google"}] }</pre>

2. Architecture attendue

Votre solution devra comporter une petite interface utilisateur, un backend exposant une API et un serveur LLM local. Le frontend ne doit pas interroger directement le modèle : les appels passent par le backend.

Interface saisie du texte, paramètres, affichage du résultat	Backend / API construction du prompt, appel au modèle, validation	LLM local Llama 3.2 1B via Ollama, llama.cpp, etc.
--	---	--

3. Travail demandé

1. Mettre en place l'appel au modèle local depuis votre backend.
2. Réaliser un premier système zero-shot simple.
3. Améliorer le prompt en ajoutant du contexte et des instructions précises sur les relations à extraire.
4. Utiliser du few-shot prompting avec quelques exemples bien choisis.
5. Tester une stratégie de raisonnement ou de décomposition avant la réponse finale.
6. Comparer plusieurs valeurs de température.

-
7. Produire une sortie structurée, de préférence en JSON, et la valider côté code.
 8. Comparer les différentes stratégies sur plusieurs textes afin d'identifier celles qui améliorent réellement le système.

4. Données et évaluation

Vous travaillerez sur un jeu de données d'extraction d'informations fourni ou indiqué pendant le TP. Une partie des exemples pourra être utilisée pour construire les prompts, tandis qu'un autre ensemble servira à évaluer votre système.

Votre comparaison devra porter au minimum sur la qualité de l'extraction, la validité du format produit et la présence éventuelle d'informations inventées. Une relation ne doit être retournée que si elle est réellement supportée par le texte.

5. Contraintes

- Le modèle final doit être exécuté localement et rester de petite taille (environ 1B de paramètres).
- GPT, Claude ou d'autres grands modèles externes ne peuvent pas être utilisés comme moteur de l'application finale.
- Les différentes stratégies doivent être comparées avec le même modèle et les mêmes exemples d'évaluation.
- Pour une comparaison correcte des prompts, les paramètres de génération doivent être contrôlés et explicités.

6. Livrable attendu

À la fin du TP, vous devrez disposer d'une application fonctionnelle permettant de saisir ou sélectionner un texte, modifier certains paramètres, lancer l'extraction et visualiser le résultat.

Vous devrez également être capable d'expliquer brièvement quelle stratégie de prompting vous avez retenue et quel impact ont eu le contexte, le few-shot, le raisonnement et la température sur les résultats obtenus.

Important : l'objectif n'est pas d'obtenir un prompt qui fonctionne sur un seul exemple, mais de construire une solution qui reste robuste sur plusieurs documents.

Le lien vers le dataset à utiliser : <https://huggingface.co/datasets/tonytan48/Re-DocRED>